

P05

Estimación eficiente de representaciones de palabras en un espacio vectorial

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean · 2013 · arXiv:1301.3781 · ICLR 2013 (workshop)

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P05 — Word2Vec

El momento en que el significado de una palabra se vuelve una dirección en el espacio, y esa dirección se puede calcular a escala de miles de millones de palabras.

Nivel: L2 · Motor: word2vec · Notebook: P05_word2vec.ipynb · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Efficient Estimation of Word Representations in Vector Space</i>
Autoría	Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean
Año	2013
Venue	arXiv:1301.3781 · presentado en el taller de ICLR 2013
Fuente primaria	arXiv:1301.3781
Complemento imprescindible	arXiv:1310.4546 (muestreo negativo, NIPS 2013)
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Representar palabras como identificadores dispersos (*one-hot*) tiene una consecuencia devastadora: **todas las palabras están a la misma distancia entre sí**. «Perro» no está más cerca de «gato» que de «hipoteca». Cualquier modelo que parta de ahí tiene que aprender la similitud desde cero para cada tarea.

Existían representaciones distribuidas: el modelo neuronal de lenguaje de Bengio et al. (2003) ya producía vectores de palabras como efecto colateral. Pero incluía una capa oculta no lineal y una softmax sobre todo el vocabulario: entrenarlo sobre corpus grandes era prohibitivo.

3. Propuesta

Dos arquitecturas **log-lineales, sin capa oculta**, diseñadas para ser baratas:

- **CBOW**: predice la palabra central a partir de su contexto.
- **Skip-gram**: predice el contexto a partir de la palabra central.

El artículo complementario (Mikolov et al., 2013b) añade el ingrediente que las hace prácticas: **muestreo negativo**, que sustituye la softmax sobre todo el vocabulario por unas pocas clasificaciones binarias «esta pareja es real / esta pareja es inventada».

El resultado inesperado: el espacio resultante exhibe **estructura lineal**. Ciertas relaciones semánticas y sintácticas aparecen como desplazamientos aproximadamente constantes.

4. Intuición sin fórmulas

«Dime con quién apareces y te diré qué significas.» Si dos palabras aparecen rodeadas de las mismas palabras, acabarán con vectores parecidos — sin que nadie escriba jamás una definición.

Dónde deja de funcionar la analogía: el método captura **distribución**, no significado. Antónimos como «caliente» y «frío» aparecen en contextos casi idénticos, y por tanto quedan cerca. El espacio no distingue «similar» de «intercambiable».

5. Matemática mínima

Skip-gram con muestreo negativo. Para un par observado (centro `c`, contexto `o`) y `k` negativos `n1...nk` muestreados de una distribución de ruido:

```
L = - log σ(v_c · u_o) - Σ_i log σ(-v_c · u_{n_i})

σ(z) = 1/(1+e-z)
v_c : vector de entrada (el que se usa como embedding final)
u_o : vector de salida (contexto)
```

Similitud y analogía:

```
similitud(a, b) = cos(v_a, v_b) = (v_a · v_b) / (||v_a|| ||v_b||)

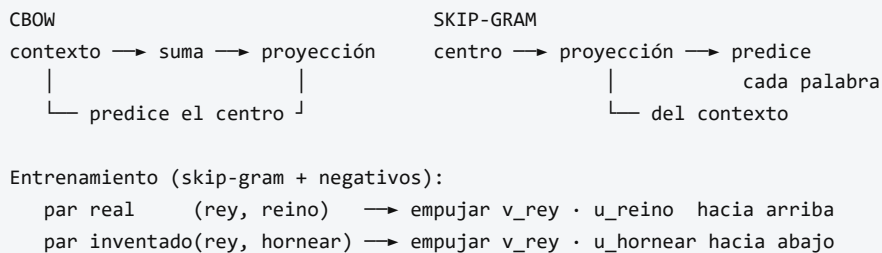
analogía a : b :: c : ? → argmax_d cos(v_b - v_a + v_c, v_d), d ∉ {a, b, c}
```

La exclusión de `{a, b, c}` del ranking **es parte del protocolo**, no un detalle de implementación: sin ella el resultado cambia por completo.

[!TIP] **Puente matemático.** Esta sección da por sabido lo siguiente. Si algo no te suena, léelo primero: está explicado una sola vez, en un solo sitio, y sirve para todas las fichas.

Dónde	Qué necesitas de ahí
A01 §2 · Norma y coseno	la similitud coseno es cómo se lee la geometría que el modelo aprende
A02 §1 · Softmax	el softmax sobre el vocabulario y por qué su coste motiva el muestreo negativo

6. Arquitectura o flujo



7. Qué observar en el paper original

- La **tabla de coste computacional**: el argumento central del artículo es de eficiencia, no de calidad. Se trata de poder entrenar sobre miles de millones de palabras.
- El **conjunto de analogías** semánticas y sintácticas que los autores construyen, y la distinción entre ambos tipos.
- La comparación de calidad frente a dimensión del vector y tamaño del corpus.
- En el artículo complementario: **muestreo negativo**, submuestreo de palabras frecuentes y detección de frases.

8. Evidencia y resultados

Los autores evalúan sobre un conjunto de preguntas de analogía del tipo «*Atenas es a Grecia lo que Oslo es a ____*» (semánticas) y «*camina es a caminando lo que nada es a ____*» (sintácticas), midiendo exactitud de la respuesta exacta.

Las cifras de exactitud por tipo de analogía, dimensión y tamaño de corpus están en las tablas del artículo. Este eje no las reproduce: verificarlas allí antes de citarlas.

La miniatura de este eje produce evidencia a escala de juguete y honesta sobre su alcance: con un corpus de 8 frases, **rey - hombre + mujer** devuelve **reina** como vecino más cercano en las tres semillas probadas, con un coseno claramente separado del segundo candidato.

9. Impacto

- Los embeddings preentrenados se convierten en el punto de partida por defecto de cualquier sistema de PLN durante varios años.
- Fundan la **búsqueda vectorial** y, con ella, todo el ecosistema de bases de datos de vectores que sostiene hoy RAG.
- La estructura lineal del espacio abre una línea de investigación sobre **sesgo**: si el espacio codifica «rey - hombre + mujer = reina», también codifica asociaciones sociales problemáticas (Bolukbasi et al., 2016; Caliskan et al., 2017).
- Popularizan la idea de que **el preentrenamiento no supervisado produce representaciones reutilizables** — la premisa completa de BERT y GPT.

10. Limitaciones

1. **Un vector por palabra.** «Banco» tiene una única representación para el asiento, la entidad financiera y la orilla. Lo resolverán ELMo y BERT con embeddings contextuales.
2. **Sin composición.** No hay forma principitada de obtener el vector de una frase.
3. **Vocabulario cerrado.** Una palabra no vista no tiene vector (lo abordará FastText con subpalabras).
4. **Captura distribución, no significado.** Antónimos quedan cerca.
5. **Hereda y amplifica sesgos** del corpus.
6. **La aritmética de analogías es más frágil de lo que sugiere su fama:** depende del protocolo de exclusión, de la normalización y del subconjunto evaluado.

11. Errores comunes

Error	Corrección
«Word2Vec inventó los embeddings»	Bengio et al. (2003) ya producía vectores de palabras. Word2Vec los hizo baratos a gran escala.
«rey - hombre + mujer = reina es una igualdad»	Es el vecino más cercano de un vector calculado, tras excluir las tres palabras de la consulta. No es una identidad algebraica.
«El muestreo negativo está en el paper de enero de 2013»	Está en el artículo complementario de octubre de 2013 (arXiv:1310.4546).
«Word2Vec entiende el significado»	Modela co-ocurrencia. Que la geometría resultante sea interpretable no implica comprensión.
«Los embeddings son neutrales porque los aprende una máquina»	Reflejan el corpus. La neutralidad no es un efecto secundario del automatismo.

12. Relación con trabajos anteriores

- **Harris (1954), Firth (1957)** — hipótesis distribucional.
- **Bengio et al. (2003)** — modelo neuronal de lenguaje con representaciones distribuidas.
- **LSA / análisis semántico latente (1990)** — factorización de matrices de co-ocurrencia.

13. Relación con trabajos posteriores

- **GloVe (Pennington et al., 2014)** — enfoque basado en factorizar co-ocurrencias globales. ACL Anthology
- **FastText (2016)** — subpalabras; resuelve el vocabulario cerrado.
- **ELMo (2018)** — embeddings **contextuales**: un vector distinto por aparición. ACL Anthology
- **P09 BERT (2018)** — representaciones contextuales profundas.
- **P11 RAG (2020)** — recuperación densa apoyada en embeddings.

14. Notebook asociado

P05_word2vec.ipynb

Qué implementa: skip-gram con muestreo negativo entrenado desde cero en Python puro sobre un corpus de 8 frases, con vecinos por coseno y evaluación de analogía con protocolo de exclusión explícito.

Qué NO implementa: CBOW, submuestreo de frecuentes, detección de frases, ni ninguna evaluación a escala. Con 21 palabras de vocabulario, todo resultado es ilustrativo.

```
ai-evolution paper-lab P05 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida de skip-gram con muestreo negativo y explica cada término.
Explicar	Explica por qué el muestreo negativo es más barato que la softmax completa.
Aplicar	Ejecuta el notebook con tres semillas y anota qué cambia y qué se mantiene.
Analizar	Quita la exclusión de {a, b, c} del ranking de analogías y explica el resultado.
Evaluar	Un artículo afirma que los embeddings «demuestran que el modelo entiende relaciones de género». Evalúa la afirmación.
Crear	Construye 10 analogías propias en español, ejecútalas y clasifica los fallos por tipo (frecuencia, polisemia, corpus).

16. Autoevaluación

1. ¿Qué diferencia hay entre CBOW y skip-gram, y cuándo conviene cada uno?
2. ¿Por qué la softmax completa es costosa y cómo lo evita el muestreo negativo?
3. ¿Por qué «caliente» y «frío» acaban cerca en el espacio?
4. ¿Qué papel juega la ventana de contexto en el tipo de similitud que se aprende?
5. ¿Por qué hay dos matrices de vectores (entrada y salida) y cuál se usa como embedding?
6. ¿Qué problema de la polisemia no puede resolver este método, y qué lo resolvió?
7. ¿Qué parte del método que hoy se llama «Word2Vec» no está en el paper de enero de 2013?

17. Respuestas esperadas

1. CBOW predice el centro desde el contexto (más rápido, mejor con palabras frecuentes); skip-gram predice el contexto desde el centro (mejor con palabras raras y corpus pequeños).
2. Porque normalizar exige sumar sobre todo el vocabulario en cada paso. El muestreo negativo sustituye eso por $k+1$ clasificaciones binarias, con k típicamente entre 5 y 20.
3. Porque aparecen en contextos casi idénticos. El método mide sustituibilidad distribucional, no relación semántica.
4. Ventanas pequeñas favorecen similitud sintáctica y funcional; ventanas grandes favorecen similitud temática.
5. Cada palabra juega dos papeles (centro y contexto) y necesita un vector para cada uno. Habitualmente se usa la matriz de entrada como embedding final.

6. Que una palabra tenga un único vector para todos sus sentidos. Lo resolvieron los embeddings contextuales: ELMo y después BERT.
7. El muestreo negativo, el submuestreo de palabras frecuentes y la detección de frases: todo ello está en [arXiv:1310.4546](#).

18. Fuentes primarias

- Mikolov, T., Chen, K., Corrado, G. y Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. [arXiv:1301.3781](#) · consultado 2026-08-16.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. **NIPS 2013**. [arXiv:1310.4546](#) · consultado 2026-08-16.
- Pennington, J., Socher, R. y Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. **EMNLP 2014**. [ACL Anthology](#) · consultado 2026-08-16.



Evaluación — P05 · Estimación eficiente de representaciones de palabras en un espacio vectorial

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Efficient Estimation of Word Representations in Vector Space* (2013, arXiv:1301.3781 · ICLR 2013 (workshop)) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P05_word2vec.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).